

CDF: Certified Discovery Federation

Clocked Settlement-Triggered Scout Activation, AMLD Emulation, ATP Conjecture
Generation, and Infinite-Width Discovery Geometry

Brian Tenneson

With the assistance of OpenAI

10/1/2026

Living Research Paper / Version 12

CDF is intentionally not presented as a final specification. It is a versioned research framework designed to evolve as its formal definitions, controller policies, proofs, and benchmark evidence are extended.

Abstract

We introduce the *Certified Discovery Federation (CDF)*, a verifier-gated architecture for formal discovery combining automated theorem provers, AMLD settlement roles, persistent certified memory, dynamically activated Scout processes, persistent Board state, and a Player-Zero control layer.

The organizing principle is that computational resources should expand in response to *certified epistemic progress* rather than merely elapsed search. CDF therefore distinguishes an analytical frontier measure from the discrete *credit rank* used for spawning. Let F_t be the certified settlement frontier and let

$$\gamma_t = \kappa(F_t) \in \mathbb{N}$$

for a frozen verifier-computable rank κ . A spawn credit is earned only when γ_{t+1} establishes a new certified record low relative to all earlier credit ranks. Consequently, frontier oscillation, repeated rediscovery of the same contraction, and arbitrarily small numerical fluctuations cannot farm credits. A credit creates permission rather than a process: actual activation may occur only at a scheduled Player-Zero service turn, where Player Zero may spend or defer the credit.

Scout, Board, and search-worker resources are tracked separately. Activating a Board changes persistent shared control state but does not, by itself, increase candidate-testing capacity. If an experiment couples a Board with a new worker, that coupling must be declared explicitly and both resource counters must be updated and charged separately.

The first structural result is a *Scout Emulation Theorem*. Under an explicit capability interface, a sufficiently general Scout can reproduce every finite ordinary-AMLD trace while maintaining encodings of the role-local states it must later resume. The theorem preserves legal reachability and verifier-visible output; it does not claim preservation of parallel running time, charged cost, probability law, or scheduler behavior. Extension to the full +0 control layer requires additional assumptions concerning Player Zero, Board state, scheduling authority, and persistent shared control state.

A corresponding *ESLF Transfer Corollary* shows that every finite admissible Eureka-Style Lemma Fishing (ESLF) execution, and every finite prefix of an effectively generated ESLF execution, can be reproduced by such a Scout. ESLF searches for nontrivial intermediate statements not supplied in advance while requiring verifier-checkable derivability, target recovery, and an explicit finite contradiction certificate

$$H \cup A \cup \{-T\} \vdash \perp$$

(or the declared proof-system analogue), together with a frozen effective anti-shortcut checker. Thus the Scout may search for intermediate statements that appear highly nonobvious from the original target while leaving theoremhood entirely under the fixed verifier.

CDF additionally imports the clocked-simulation antichain theorem for AMLD+0. The cited construction proves that the AMLD+0 clocked-simulation degree poset contains a countably infinite antichain represented by finite, finitely describable AMLD+0 systems, even when Player Zero performs only one tick-1 activation of a common rigid Board. Thus the ambient computational geometry available to CDF is not linearly ordered even before adaptive Scout activation is introduced.

ATP conjecture generation is deliberately treated as a general proposal interface rather than a single fixed algorithm. A concrete example developed here is *scientific hypothesis generation*: an ATP may propose formalized hypotheses and consequences, AMLD/Scouts may settle their logical obligations, the verifier may certify derivations relative to their stated assumptions, and empirical observations remain separately tracked through an external evidence interface. In particular, a consequence proved from $H \cup \{h\}$ enters BANK with its hypothesis context and provenance; it is not promoted to an unconditional theorem. Thus hypothesis generation, formal validation, and empirical confirmation are not collapsed into one operation.

The resulting architecture forms a closed discovery loop:

ATP conjecture generation $\rightarrow$ AMLD settlement $\rightarrow$ verification $\rightarrow$ BANK $\rightarrow$ new record credit-rank contraction $\rightarrow$ spawn credit $\rightarrow$ Player-Zero turn $\rightarrow$ optional declared activation $\rightarrow$ new discovery

The central research question is whether this certified feedback loop can convert difficult proof search from comparatively opaque password seeking into increasingly structured navigation without changing the verifier trust boundary; the password-seeking framing is developed separately in earlier work [5].

Contents

1	Introduction	5
2	The CDF architecture	5
3	The settlement frontier	6
4	Clock-respecting settlement-triggered activation	7
5	Ordinary Scouts versus Graduate Scouts	9
6	A general Scout	9
7	Scout Emulation Theorem	9
8	What the theorem does not yet prove	10
9	Eureka-Style Lemma Fishing (ESLF)	11
10	ESLF Transfer Corollary	13
11	Miracle-looking lemmas	14
12	ATPs as conjecture generators	14
12.1	Scientific hypothesis generation as ATP conjecture generation	15
13	The complete feedback cycle	17
14	Hidden-target basin control and marginal deployment	17
14.1	Questions that should control the search	18
14.2	A basin detector without Board-overlap statistics	18
14.3	The brute-force baseline	19
14.4	Lemma value versus worker value	20
14.5	Basin-mode control	20
14.6	A compact Player-Zero fishing prompt	21
15	Querying our way to an elusive proof: an Ocean-style blind proxy	21
15.1	Reproducible blind hidden-target proxy	22
15.2	A simple crossover law	24
16	Clocked-simulation antichains in AMLD+0	24
17	Permutation Reflection	25
18	AMLD+0 Infinite-Width Theorem	26
19	Why the antichain matters for CDF	26

20 A first CDF research program	27
20.1 Scout conservativity	27
20.2 AMLD+0 Scout simulation	27
20.3 Frontier correctness	27
20.4 Ordinary versus Graduate Scout activation	27
20.5 ESLF productivity	27
20.6 Adaptive value theorem	27
21 Central Principle	27
22 Conclusion	28
A Installing an AMLD+0 Reference Instance	28
A.1 What “install AMLD+0” means	28
A.2 Before installation: freeze the experiment	29
A.3 Minimal component layout	29
A.4 Reference installation procedure	30
A.5 Required self-test	31
A.6 A minimal run after installation	32
A.7 In plain English	32
A.8 Installation checklist	32
References	33
Suggested Further Reading	34

1 Introduction

Automated theorem proving is usually organized around a fixed collection of provers operating on a fixed target. Additional resources may be supplied by running more provers, increasing search budgets, changing premise selection, or supplying intermediate lemmas.

CDF introduces a different organizing principle. The federation may change its own discovery capacity in response to *certified changes in the geometry of the problem being settled*.

The key distinction is

$$\boxed{\text{more computation} \neq \text{more certified knowledge.}}$$

CDF therefore does not activate new discovery resources merely because a certain number of CPU cycles have elapsed or because a search has become difficult. Instead, a certified event in the settlement structure becomes a control signal. Schematically,

$$\text{certified progress} \implies \text{spawn opportunity for Player Zero.}$$

This produces a feedback architecture in which proof discovery changes the state of the discovery organization itself.

The central components draw on the AMLD/AMLD+0 lineage, five-role cooperative settlement, and verified-BANK geometry developed in earlier work [6, 12, 11]. They are:

- (1) one or more ATP engines;
- (2) the AMLD roles P, R, I, D, C ;
- (3) a fixed verifier V ;
- (4) a certified shared BANK;
- (5) ordinary or Graduate Scouts;
- (6) an active settlement frontier;
- (7) Player Zero as scheduler and activation controller.

The trust boundary remains fixed:

$$\boxed{\text{search processes may propose freely; only } V \text{ certifies knowledge.}}$$

Thus CDF is intended to increase discovery power without granting any discovery process authority to declare its own output true. This separation between discovery and certification follows the verifier-gated viewpoint developed in earlier AMLD work [17, 18].

2 The CDF architecture

At time t , let the federation/controller state be

$$X_t = (P_t, R_t, I_t, D_t, C_t, E_{1,t}, \dots, E_{e_t,t}, S_{1,t}, \dots, S_{m_t^S,t}, \text{BANK}_t, \mathcal{B}_t, F_t, P_{0,t}),$$

where:

- P is the proving role;

- R is the refuting role;
- I investigates independence;
- D investigates dependence, reduction, or structural implication;
- C investigates contradiction;
- $E_1, \dots, E_{e_t}$ are ATP or conjecture-generating engines;
- $S_1, \dots, S_{m_t^S}$ are active Scouts;
- BANK_t is verifier-authorized certified memory;
- $\mathcal{B}_t$ is persistent Board/control state, logically distinct from BANK_t ;
- F_t is the active certified settlement frontier;
- P_0 is Player Zero.

When candidate-testing or other worker capacity matters, CDF tracks it with a separate counter m_t^W . Likewise, if several Boards are active, their count may be written m_t^B . No identity such as $m_t^B = m_t^W$ is assumed. An experiment may declare a coupled Board–worker package, but that coupling is then part of the frozen protocol and both costs are charged.

Not every component must be active at every time. Player Zero may allocate service among enabled components, subject to the declared protocol.

An ATP may propose a candidate formula L . That formula is not thereby promoted to knowledge. Instead,

$$E_i \text{ proposes } L$$

creates a settlement obligation such as

$$\text{Settle}(L).$$

AMLD or Scout processes may then investigate L through the relevant channels. Only verifier-authorized outcomes may alter certified BANK. Hence

$$\boxed{\text{proposal} \rightarrow \text{settlement attempt} \rightarrow \text{verification} \rightarrow \text{certified state.}}$$

3 The settlement frontier

Let F_t denote the active certified settlement frontier at time t . The exact representation of F_t may depend on the formal environment. It may consist of minimal unsettled obligations, minimal certified packages, boundary states, or another declared finite representation.

CDF permits two related but distinct frontier summaries. First, an analytical measure

$$r_t = \rho(F_t)$$

may be used for diagnosis, value estimation, or basin detection. Second, when settlement-triggered spawning is enabled, the experiment must freeze a verifier-computable *credit rank*

$$\gamma_t = \kappa(F_t) \in \mathbb{N}.$$

The credit rank is the quantity used by the spawn rule. Requiring a discrete well-founded rank prevents arbitrarily small real-valued fluctuations from creating unbounded activation rights.

We call

$$r_{t+1} < r_t$$

an *analytical frontier contraction* when the inequality follows entirely from verifier-authorized information. We call

$$\gamma_{t+1} < \min_{0 \leq u \leq t} \gamma_u$$

a *new-record certified credit contraction*. Only the latter earns a spawn credit in the baseline CDF policy. The surrounding language of settlement horizons, fair settlement geometry, and finite-breakthrough structure is motivated by earlier treatments [9, 10, 13].

The distinction between finding a lemma and contracting the frontier is essential. A newly certified lemma may be interesting without changing either summary:

$$L \in \text{BANK}_{t+1} \setminus \text{BANK}_t$$

does not necessarily imply

$$r_{t+1} < r_t \quad \text{or} \quad \gamma_{t+1} < \min_{0 \leq u \leq t} \gamma_u.$$

CDF therefore rewards structural progress rather than merely lemma count. In particular, an oscillation such as $10 \rightarrow 9 \rightarrow 10 \rightarrow 9$ in a displayed diagnostic measure cannot earn two baseline credits unless the second event establishes a genuinely new discrete record under the frozen credit rank.

4 Clock-respecting settlement-triggered activation

A new-record certified credit contraction does not interrupt the current AMLD+0 schedule and does not immediately create a Scout, Board, or worker. Instead, it creates a pending right for Player Zero to consider one declared activation when the global clock next assigns service to Player Zero.

Definition 4.1 (Clocked service function). Let

$$\chi(t) \in \{P, R, I, D, C, P_0, \dots\}$$

denote the process whose turn is serviced at global clock tick t . A Player-Zero activation action is legal only at a tick s satisfying

$$\chi(s) = P_0.$$

Definition 4.2 (Spawn-credit counter). Let $q_t \in \mathbb{N}$ be the number of pending spawn credits held by Player Zero, and define the running certified record

$$\gamma_t^* = \min_{0 \leq u \leq t} \gamma_u.$$

The running record is scoped to the frozen target/activation epoch. It is reset only when the protocol explicitly begins a new epoch; merely adding obligations, changing search policy, or seeing the rank rise does not reset it.

One spawn credit is earned at transition $t \rightarrow t + 1$ exactly when

$$\gamma_{t+1} < \gamma_t^*.$$

Then

$$q_{t+1} = q_t + 1,$$

subject only to bookkeeping needed when a credit-spending event is also processed at the same logical boundary. Equal or higher ranks earn no credit.

Thus the baseline law is

$$\boxed{\gamma_{t+1} < \gamma_t^* \implies \text{earn one spawn credit.}}$$

Because $\gamma_t \in \mathbb{N}$ and only new record lows count, neither repeated oscillation nor infinitesimal contraction can farm credits. A different experiment may award multiple credits for crossing multiple discrete rank levels, but it must freeze that rule in advance.

At a later Player-Zero service turn s with $q_s > 0$, Player Zero chooses

$$a_s \in \{\text{activate, defer}\}.$$

If activation is chosen, the experiment must also name the activation target

$$z_s \in \{\text{Scout, Board, search worker, declared coupled package}\}.$$

One credit is spent, but the resource counters change according to the target. For example,

$$\begin{aligned} z_s = \text{Scout} : \quad (q_s, m_s^S) &\mapsto (q_s - 1, m_s^S + 1), \\ z_s = \text{Board} : \quad (q_s, m_s^B) &\mapsto (q_s - 1, m_s^B + 1), \\ z_s = \text{search worker} : \quad (q_s, m_s^W) &\mapsto (q_s - 1, m_s^W + 1). \end{aligned}$$

A Board activation alone does *not* imply $m_{s+1}^W = m_s^W + 1$. If a Board-associated worker is part of a frozen coupled package, both Board and worker counters are incremented and both costs are charged.

If Player Zero defers, the credit and all resource counts remain unchanged. Pending credits remain available for later Player-Zero turns unless the declared experiment specifies a frozen expiration rule. When deactivation is disallowed, each resource count is monotone in the obvious coordinate.

Using an integer counter rather than a Boolean flag preserves distinct *new-record* contractions that occur before Player Zero receives service, without rewarding repeated returns to a previously reached frontier level.

Theorem 4.3 (Clock-respecting Activation Admissibility). *Suppose CDF operates under a fixed scheduled clock and the frozen credit-rank rule above. New-record certified credit contractions generate spawn credits, while resource creation is permitted only during Player-Zero service turns. Then settlement-triggered activation can be implemented without preempting or modifying the service turn of any ordinary AMLD role.*

Proof. A qualifying credit contraction changes only certified controller state by updating the record rank and pending-credit counter. It does not itself add a process and does not alter the role currently receiving service. Actual activation can occur only at a tick s satisfying $\chi(s) = P_0$. At that service boundary, activation is an ordinary legal Player-Zero action. Therefore no P , R , I , D , or C service turn is displaced merely because certified progress occurred. $\square$

The resulting logical separation is

$$\boxed{\text{certified progress event} \neq \text{permission to act} \neq \text{decision to act.}}$$

The new record creates permission, the global clock determines when that permission is exercisable, and Player Zero decides whether and how to exercise it.

5 Ordinary Scouts versus Graduate Scouts

At present, CDF intentionally leaves unresolved which kind of process Player Zero should activate when a pending spawn credit is exercised on a legal Player-Zero turn: an ordinary Scout or a Graduate Scout. This is not merely terminological.

An ordinary Scout may begin from the common certified state and execute the base discovery policy. A Graduate Scout may instead inherit some additional certified information, policy state, learned search structure, or previously established navigation strategy.

These give distinct policies

$$\pi_{\text{ordinary}} \quad \text{and} \quad \pi_{\text{graduate}}.$$

A future Player-Zero rule may compare their charged continuation values,

$$Q(X_t, \text{activate ordinary})$$

and

$$Q(X_t, \text{activate graduate}),$$

and select the larger value when the comparison is computable from legally available state.

CDF therefore treats Scout type as an *open control parameter*.

6 A general Scout

Definition 6.1 (AMLD-capable Scout). A Scout S is *AMLD-capable* if it has access to the same admissible task descriptions, observation interfaces, local computational operations, and verifier queries available to the ordinary AMLD roles; can maintain an internal role tag

$$J \in \{P, R, I, D, C\};$$

and can maintain a finite effective encoding

$$\mu = (\mu_P, \mu_R, \mu_I, \mu_D, \mu_C)$$

of every legally visible role-local state that may need to be resumed later in a sequential simulation. When operating under tag J , the Scout applies the transition policy assigned to AMLD role J to the encoded local state μ_J and updates that coordinate accordingly.

This definition gives the Scout no privileged information. In particular, the Scout receives no hidden theorem, proof, settlement certificate, oracle output, verifier bypass, scheduler authority, or Board state unavailable under the declared role interface.

7 Scout Emulation Theorem

Theorem 7.1 (AMLD Scout Emulation). *Let*

$$\mathcal{A} = (P, R, I, D, C)$$

be an AMLD federation whose role transitions are effective under a common admissible task interface. Let S be an AMLD-capable Scout. Then S can perform every individual task executable by any AMLD role.

Moreover, for every finite AMLD execution

$$x_0 \xrightarrow{J_1} x_1 \xrightarrow{J_2} \dots \xrightarrow{J_n} x_n,$$

where each $J_i \in \{P, R, I, D, C\}$, there exists a sequential Scout execution that reproduces the same legal role-local transition sequence and verifier-visible outputs by successively assuming role tags

$$J_1, J_2, \dots, J_n$$

and updating the corresponding encoded role-local states.

Proof. For a single AMLD action executed by role J , the Scout selects tag J , reads the encoded legally visible role-local state μ_J , and applies the same effective transition procedure. It then stores the resulting legal local state back into μ_J . Hence every individual AMLD transition can be reproduced.

For a finite AMLD trace, proceed by induction on trace length. The empty trace is immediate. Assume the Scout has reproduced the first n transitions while preserving effective encodings of the role-local states required for later resumption. For transition $n + 1$, select tag J_{n+1} , restore $\mu_{J_{n+1}}$, apply the corresponding transition rule, record the verifier-visible output, and update that local-state coordinate. Thus the next transition is reproduced. Finite induction completes the simulation. $\square$

Remark 7.2 (What is preserved). The theorem is a reachability and interface-emulation statement. It preserves the finite legal trace and verifier-visible outputs. It does *not* assert equality of parallel wall time, charged computational cost, random-output distribution, fairness guarantees, or scheduling behavior between a multi-role federation and one sequential Scout. Any stronger resource-preservation theorem requires additional assumptions.

8 What the theorem does not yet prove

The preceding theorem concerns AMLD. It does not automatically prove that one Scout can simulate all of AMLD+0.

Player Zero adds functionality that need not belong to an ordinary AMLD role:

- global scheduling;
- persistent Board state;
- activation authority;
- federation-level observation;
- resource allocation;
- policy selection;
- coordination across simultaneously active processes.

Thus

Scout simulates AMLD

does not yet imply

Scout simulates AMLD+0.

An AMLD+0 Scout-simulation theorem would require an additional condition giving the Scout a legal representation of Player-Zero state and control authority. This remains open in the present paper.

9 Eureka-Style Lemma Fishing (ESLF)

A central use of a general Scout is not merely to attack the headline theorem directly. The Scout may instead search for statements whose relevance is not initially obvious.

The general idea of automated lemma discovery is not claimed here as new. Earlier automated-reasoning work explicitly studied “eureka steps,” failure-guided lemma discovery, and the automatic search for missing intermediate results [2, 3]. The term *Eureka-style* acknowledges that lineage. In CDF, *Eureka-Style Lemma Fishing (ESLF)* names the more specific verifier-gated, certificate-shaped protocol defined below.

Let the target be T and let H be the frozen set of admitted hypotheses. ESLF searches for intermediate candidate statements

$$L_1, L_2, \dots$$

without specifying in advance which statements will be useful. The search is not scored merely by the production of plausible lemmas. A successful ESLF output must satisfy a fixed certificate interface.

Definition 9.1 (Eureka-Style Lemma Fishing (ESLF)). Fix a target statement T , a frozen hypothesis set H , a fixed verifier V , and a predeclared effective anti-shortcut checker $\text{NT}_{H,T}$. Its implementation and charged cost are part of the frozen protocol. An *ESLF certificate* is a triple

$$(n, A, \Pi), \quad n > 0, \quad A = \{A_1, \dots, A_n\}, \quad |A| = n,$$

where A is a finite nonempty family of well-formed statements and Π contains verifier-checkable proof data, including derivations π_i , a target derivation π_T , and an explicit contradiction certificate $\pi_\perp$, satisfying:

- (i) for each i ,

$$V(H \vdash A_i, \pi_i) = 1;$$

- (ii) the target is derivable once the fished lemmas are available,

$$V(H \cup A \vdash T, \pi_T) = 1;$$

- (iii) the target-negated extension has an explicit finite verifier-accepted contradiction certificate,

$$\boxed{V(H \cup A \cup \{\neg T\} \vdash \perp, \pi_\perp) = 1};$$

if the proof system uses a designated inconsistency object other than $\perp$, that declared object is used instead;

- (iv) all certificate components are checked by the fixed verifier rather than by the proposing agent; and

- (v) the proposed decomposition passes the frozen effective anti-shortcut checker,

$$\text{NT}_{H,T}(A, \Pi) = 1.$$

The word *fishing* emphasizes that useful intermediate lemmas are not supplied in advance. The word *Eureka-style* emphasizes that their relevance may become clear only after discovery and certification.

For Metamath-facing discovery, $\vdash$ is specialized to $\vdash_{\text{MM}}$, every A_i must be a well-formed formula in the frozen Metamath language, and the displayed obligations must be accompanied by the corresponding verifier-checkable Metamath proof objects. The contradiction obligation must likewise be represented by an explicit object accepted by the frozen Metamath-facing verification layer; no semantic “consistency oracle” is assumed.

The point of the definition is that the fished lemmas are not new assumptions. They are themselves consequences of H and are used only as intermediate certified structure.

Proposition 9.2 (ESLF certificate soundness). *If an ESLF certificate (n, A, Π) exists for a Metamath-facing instance (H, T) , then*

$$\boxed{H \vdash_{\text{MM}} T.}$$

Proof. For every $A_i \in A$, condition (i) supplies a verifier-accepted derivation of A_i from H . Condition (ii) supplies a derivation of T from $H \cup A$. Compose these derivations by substituting the H -derivation of each A_i wherever that lemma is invoked. The resulting proof depends only on H , and therefore $H \vdash_{\text{MM}} T$.

Condition (iii) supplies a separate finite contradiction witness for $H \cup A \cup \{\neg T\}$. It is not logically needed for the preceding derivability implication once (i) and (ii) have been certified. In ESLF the apparent redundancy is intentional: the contradiction witness is a first-class verifier-checkable settlement object rather than a metamathematical assertion of non-consistency. $\square$

Thus the characteristic ESLF problem is not merely

“find a lemma related to T .”

It is the more constrained search problem

$$\boxed{\begin{aligned} &\text{find a finite nonempty } A \text{ and proof data } \Pi \text{ such that} \\ &V(H \vdash A_i, \pi_i) = 1 \text{ for every } A_i \in A, \\ &V(H \cup A \vdash T, \pi_T) = 1, \\ &V(H \cup A \cup \{\neg T\} \vdash \perp, \pi_\perp) = 1, \\ &\text{NT}_{H,T}(A, \Pi) = 1. \end{aligned}}$$

Definition 9.3 (Frozen anti-shortcut admissibility). For an ESLF task (H, T) , let

$$\text{NT}_{H,T}(A, \Pi) \in \{0, 1\}$$

be a frozen, effective, terminating checker on the proposed lemma set A and associated certificate data Π . Its code or complete decision rule is fixed before a scored run, and its runtime is charged. An ESLF certificate is admissible only if $\text{NT}_{H,T}(A, \Pi) = 1$.

The checker may exclude predeclared classes of trivial or answer-bearing settlements: taking a fished lemma to be T itself, using a merely syntactic or definitionally immediate copy of T , encoding a hidden successful proof or answer in a proposed lemma, or exploiting prohibited target metadata rather than discovering intermediate structure. These examples are illustrative rather than exhaustive.

In practice there may be many bailout families. The exact list can depend on the formal environment and benchmark, but the admissibility rule must be fixed *before* a scored ESLF run. A benchmark may therefore refer to a separately versioned exclusion specification

$$\mathcal{F}_{\text{triv}}^{(v)}$$

whose members are the forbidden shortcut families for version v . The scored condition is then

$$(n, A, \Pi) \text{ is accepted} \iff \text{the proof objects verify and } \text{NT}_{H,T}^{(v)}(A, \Pi) = 1.$$

This prevents both obvious bailout behavior by AMLD+0 and post hoc movement of the nontriviality standard by the experimenter.

Remark 9.4 (Proposed distinctive feature and priority scope). CDF does not claim priority for automated lemma discovery, eureka-step reasoning, proof critics, or theory exploration in general. The proposed distinctive feature of ESLF is the *certificate-shaped conjunction* of: (a) nontrivial intermediate lemmas derivable from the original hypotheses; (b) recovery of the target from those lemmas; (c) a separate explicit finite verifier-accepted contradiction certificate for $H \cup A \cup \{\neg T\}$; (d) verifier acceptance; and (e) a predeclared effective anti-shortcut checker. The author regards this explicit certificate interface as a candidate original contribution of ESLF. This paper does not make a categorical historical-priority claim without an exhaustive prior-art search.

The anti-shortcut condition is deliberately separate from logical soundness. Conditions (i)–(iv) govern proof correctness; NT determines whether a sound certificate counts as a successful *ESLF discovery*. Thus ESLF asks not merely for a proof decomposition, but for a nontrivial discovered intermediate decomposition carrying an explicit settlement witness.

Examples include instructions of the form:

- search for an intermediate statement whose verification would reduce the active settlement frontier;
- search for a lemma joining two currently disconnected certified regions;
- search for a dependency that would collapse several settlement obligations;
- search for a contradiction certificate for a competing branch;
- search for a statement whose addition to BANK changes the reachable settlement geometry;
- search for a finite family of candidate lemmas and submit each to AMLD settlement.

In CDF, ESLF is closely related to the cartographic viewpoint in which proof search is treated as navigation through certified decision geometry rather than as undifferentiated enumeration [14, 15, 16].

10 ESLF Transfer Corollary

Corollary 10.1 (ESLF Transfer). *Let S satisfy the AMLD Scout Emulation Theorem. Let Φ be any finite ESLF execution consisting of legal AMLD tasks. Then S can reproduce Φ . More generally, if Φ is an effectively generated infinite or open-ended procedure, S can reproduce every finite prefix of Φ .*

Consequently, any verifier-certified ESLF output that AMLD produces after finitely many steps can also be produced by Scout execution of the corresponding finite prefix, subject to the same verifier and admissibility checker.

Proof. A finite ESLF execution is a finite AMLD-compatible trace, so the Scout Emulation Theorem applies directly. An effectively generated open-ended procedure is handled prefix by prefix. Any concrete certificate is emitted at a finite stage, hence lies in one such finite prefix. Verifier behavior is unchanged, so the same proof objects receive the same acceptance decisions. $\square$

11 Miracle-looking lemmas

The preceding corollary should not be read as saying that a Scout magically proves arbitrary lemmas. The opposite interpretation is intended.

Suppose a difficult theorem T initially appears to require an improbable intermediate statement L . From the perspective of a human or a baseline ATP, discovering L may look like a miracle.

CDF replaces

“How could anyone guess L ?”

with

“Can a legal discovery policy search a structured space in which L becomes reachable?”

Thus an ESLF prompt does not provide the miracle lemma. It defines a search process intended to *fish for* statements of the appropriate structural type, consistent with the broader program of certified formal discovery and adaptive settlement [7, 19].

The trust boundary remains

$$V(L, \pi_L) = 1.$$

Only a statement accompanied by an accepted certificate π_L may alter certified BANK or settlement geometry.

The important consequence is therefore not

Scout $\Rightarrow$ miracles.

It is

Scout + structured fishing + verification
 $\Rightarrow$ a mechanism for discovering unexpectedly useful lemmas

12 ATPs as conjecture generators

CDF need not require an ATP to prove every statement it proposes. An ATP E_i may have two logically separate outputs:

ProofCandidate(T)

and

Conjecture(L).

The second output can be particularly valuable. A proposed conjecture enters an untrusted proposal pool

$\mathcal{C}_t$.

AMLD or Scouts then attempt to classify or settle each proposal.

Thus CDF separates

conjecture generation

from

conjecture certification.

This permits aggressive conjecture generation without weakening logical trust.

12.1 Scientific hypothesis generation as ATP conjecture generation

Scientific hypothesis generation is a plausible non-ATP-facing application of the same proposal–settlement–verification architecture. The correspondence is not that a theorem prover may declare an empirical hypothesis true. Rather, an ATP or learned generator may *propose* a formal hypothesis, derive formal consequences relative to it, and hand those consequences to AMLD/Scouts and the verifier. Empirical observations enter through a separately declared evidence interface.

The central separation is

$$\boxed{\text{hypothesis generation} \neq \text{formal validation} \neq \text{empirical confirmation.}}$$

Let H be the frozen formal background theory, let O_t be the currently admitted observation/evidence ledger, let Q be an unresolved scientific question, let BANK_t be certified formal memory, let F_t be the active settlement frontier, and let $\mathcal{E}$ denote the declared empirical-evidence interface.

A crucial typing rule is that BANK records the context under which a theorem was proved. A branch-relative entry may be represented as

$$(P; \Gamma; \pi; \text{provenance}) \in \text{BANK}_t,$$

meaning that verifier V accepted π as a derivation $\Gamma \vdash P$. Thus

$$H \cup \{h\} \vdash P$$

licenses a BANK entry relative to $H \cup \{h\}$ (or, when supported by the proof system, an unconditional theorem $H \vdash h \rightarrow P$). It does *not* license unconditional insertion of P under H alone. Empirical records remain in O_t with provenance and uncertainty metadata rather than becoming formal theorems merely because they agree with a prediction.

CDF Scientific Hypothesis-Generation Algorithm. The following is an implementation-neutral CDF procedure; it specifies information flow and the trust boundary without requiring one particular conjecture generator.

Step 1. Freeze the scientific language and evidence interface. Fix a language $\mathcal{L}$ in which hypotheses and predictions are expressed. Record which statements are formal axioms or certified results and which are empirical observations supplied through $\mathcal{E}$.

Step 2. Construct the legally observable generation state. Form

$$X_t = (H, O_t, Q, \text{BANK}_t, F_t).$$

A conjecture generator may inspect only information authorized by the declared experiment.

Step 3. Generate candidate hypotheses. Each admitted generator G_i proposes candidates

$$G_i(X_t) \rightsquigarrow h_{i,1}, h_{i,2}, \dots$$

Generation may use symbolic transformations, analogy, learned guidance, Graduate-Scout inherited policy, ESLF-oriented proposal rules, or other declared methods. Generation alone confers no truth status:

$$\text{Generate}(h) \not\Rightarrow h \text{ is true.}$$

Step 4. Apply admissibility and anti-shortcut filters. Reject malformed, prohibited, hidden-answer-bearing, duplicate, or otherwise inadmissible candidates. Surviving hypotheses enter an untrusted scientific proposal pool $\mathcal{C}_t^{\text{sci}}$.

Step 5. Generate formal consequences and intermediate structure. For a surviving h , ask what follows from $H \cup \{h\}$. Candidate predictions may have the form

$$H \cup \{h\} \vdash P.$$

When appropriate, ESLF may search for nontrivial intermediate lemmas connecting the hypothesis to a prediction. The ESLF obligations, including the explicit finite contradiction certificate when that interface is invoked, remain unchanged.

Step 6. Create AMLD settlement obligations. Submit appropriate tasks to the federation: proof, refutation, dependence, independence, contradiction, consistency-relative analysis, or comparison of competing hypotheses. A proposed hypothesis is something to investigate, not something the system is authorized to believe merely because it was generated.

Step 7. Verify and type every formal result. Only verifier-accepted deductions enter certified BANK, and every entry retains its assumption context:

$$V(\Gamma \vdash P, \pi) = 1 \implies (P; \Gamma; \pi; \text{provenance}) \in \text{BANK}_{t+1}.$$

A result proved only from $H \cup \{h\}$ remains hypothesis-relative unless a separate proof discharges h .

Step 8. Compare testable predictions with empirical evidence. If a formally derived prediction P is empirically testable, query the declared evidence process $\mathcal{E}$. Record formal status and empirical status separately. For example,

$$H \cup \{h\} \vdash P$$

is a formal statement about a model or hypothesis branch, whereas “the observation is compatible with P within the declared measurement rule” is an empirical record. Agreement does not, by itself, deductively certify h .

Step 9. Generate discriminating predictions when hypotheses compete. If h_1 and h_2 both remain viable, search for a prediction or experiment that separates them. An especially clear formal target is a D for which

$$H \cup \{h_1\} \vdash D, \quad H \cup \{h_2\} \vdash \neg D.$$

The resulting D identifies an empirical question capable of distinguishing the two candidates, provided D has an admissible observational interpretation.

Step 10. Update the typed ledgers and the next generation state. Verifier-certified deductions update context-tagged BANK; empirical outcomes update O_t with provenance; the active frontier is recomputed; and the revised state becomes input to the next generation round. If the frozen discrete credit rank establishes a new record low, the ordinary CDF spawn-credit rule applies.

Thus the scientific version of the CDF cycle is

generate hypothesis $\rightarrow$ derive branch-relative predictions $\rightarrow$ settle $\rightarrow$ verify
compare with evidence $\rightarrow$ update typed state $\rightarrow$ generate again.

The same algorithm in plain English

Give the federation what is already known, what has actually been observed, and the scientific question being investigated. Let an ATP or learned generator invent candidate explanations. Do not treat those candidates as facts. Ask AMLD and the Scouts what each candidate logically implies, and make the verifier check every formal derivation. Store each deduction together with the assumptions it used. When a consequence is something the world can answer, compare it with observation or experiment through the separate empirical interface. An observation may support or challenge a hypothesis under a declared evidential rule, but it does not turn the hypothesis into a theorem. If several hypotheses survive, ask for a new prediction that would make them disagree. Feed the verified, context-tagged deductions and the new observations back into the next round.

13 The complete feedback cycle

A CDF cycle can now be represented as

$$\text{BANK}_t \longrightarrow \mathcal{C}_t \longrightarrow \text{AMLDSettle}(\mathcal{C}_t) \longrightarrow \text{BANK}_{t+1}.$$

The updated certified state induces a new frontier F_{t+1} . If the frozen credit rank satisfies

$$\gamma_{t+1} < \gamma_t^* = \min_{0 \leq u \leq t} \gamma_u,$$

then one spawn credit is added to Player Zero's pending credit counter. No process is created at that instant. At a later tick s satisfying $\chi(s) = P_0$, Player Zero may spend one pending credit on one declared activation target or may defer. Scout count, Board count, and worker capacity are updated in separate coordinates.

Hence

$\begin{aligned} &\text{BANK}_t \rightarrow \text{conjectures} \rightarrow \text{settlement} \rightarrow \text{BANK}_{t+1} \\ &\rightarrow \text{new record credit-rank contraction} \rightarrow \text{spawn credit} \rightarrow \text{wait for } P_0 \text{ turn} \\ &\rightarrow \text{optional declared activation} \rightarrow \text{new conjectures} \end{aligned}$
--

This is the defining clock-respecting CDF feedback loop.

14 Hidden-target basin control and marginal deployment

The preceding feedback loop becomes more concrete in a hidden-parameter search problem. Suppose there is a finite candidate domain

$$K_0 = \{1, 2, \dots, B - 1\}$$

and exactly one hidden parameter $b \in K_0$ is associated with the target instance $F(b)$. The system is not given b . It may only use legally observable, verifier-authorized information. Let $K_t \subseteq K_0$ be the surviving certified candidate set at time t , and let

$$N_t = |K_t|.$$

An ESLF procedure should therefore be judged in part by how much certified structure it discovers and how much it reduces N_t , rather than by how many isolated candidate values it tests.

14.1 Questions that should control the search

At a legal Player-Zero service turn, the controller should ask at least the following questions:

- (1) How many candidate values remain, i.e. what is N_t ?
- (2) How much certified settlement-frontier contraction has recent charged work produced?
- (3) How much reusable certified structural information has recent charged work produced?
- (4) Has the system entered a persistent low-contraction regime that should be treated as a candidate basin?
- (5) How many ATP workers are active, and how much nonduplicated candidate-testing capacity do they supply?
- (6) What is the measured mean charged time $\hat{\tau}_t$ for one local candidate test?
- (7) If the next unit of budget is spent on lemma fishing, what reduction $\widehat{\Delta N}_{\text{fish}}$ in the surviving candidate set is reasonably expected?
- (8) If the next legal activation opportunity is used to add a search worker (possibly as part of a declared Board-worker package), what reduction in expected remaining settlement time is obtained instead?

These questions make the controller choose between discovering structure and purchasing additional local search capacity rather than merely asking whether it “feels stuck.”

14.2 A basin detector without Board-overlap statistics

No overlap or agreement statistic between Boards is required. Let W_t denote cumulative charged work. Over a declared window of length h , define the frontier-contraction yield

$$\lambda_t = \frac{\max\{0, r_{t-h} - r_t\}}{W_t - W_{t-h}}.$$

To measure reusable structural discovery, let g_t be a predeclared verifier-based structural-gain functional on certified BANK state. It must reward certified reusable structure rather than raw lemma count. Define

$$\nu_t = \frac{g_t - g_{t-h}}{W_t - W_{t-h}}.$$

For fixed thresholds $\varepsilon_\lambda, \varepsilon_\nu > 0$ and a fixed persistence parameter q , define a candidate basin by

$$\boxed{\text{Basin}(t) = 1 \iff \lambda_t < \varepsilon_\lambda \quad \text{and} \quad \nu_t < \varepsilon_\nu}$$

for q consecutive windows.

Thus basin detection means that significant charged work has stopped contracting the certified settlement frontier and has stopped producing structural discoveries capable of restoring such contraction. This is intentionally different from claiming that no undiscovered structural escape exists. Basin detection is a controller signal, not an impossibility theorem.

14.3 The brute-force baseline

Assume temporarily that N candidates remain, there is exactly one target candidate, the target position is uniform over the N candidates, candidate tests are nonduplicated, and each test has mean charged duration τ .

With one ATP, the expected number of candidate tests is

$$\frac{N+1}{2},$$

so

$$\mathbb{E}[T \mid m = 1] = \frac{N+1}{2}\tau.$$

For the concrete case $B = 10^7$ with positive integer candidates $1 \leq b < B$,

$$N = B - 1 = 9,999,999,$$

and therefore one serial ATP requires

$$\mathbb{E}[T] = 5,000,000 \tau$$

under the stated uniform-target model.

Now suppose m ATPs partition the candidate tests without duplication. Write

$$N = qm + r, \quad 0 \leq r < m.$$

If testing proceeds in parallel batches of size at most m , then the exact expected number of batches is

$$\mathbb{E}[\text{batches} \mid m] = \frac{mq(q+1)/2 + r(q+1)}{N}.$$

Consequently,

$$\mathbb{E}[T \mid m] = \tau \frac{mq(q+1)/2 + r(q+1)}{N} \approx \frac{N\tau}{2m},$$

where the approximation has only an additive $O(\tau)$ batching term.

If the number M of active ATPs is random but is drawn once at the beginning of a run and remains fixed, then, under the same simplifying assumptions,

$$\mathbb{E}[T] \approx \frac{N\tau}{2} \mathbb{E}\left[\frac{1}{M}\right].$$

In general this is not equal to $N\tau/(2\mathbb{E}[M])$. Runs with few workers therefore contribute disproportionately to expected completion time.

If the worker population changes during a run, the preceding random- M formula should not be used. Instead one should track cumulative nonduplicated service capacity. In a continuous approximation, if $m(s)$ ATPs are active at time s , then the number of candidate tests supplied by time t is approximately

$$C(t) = \frac{1}{\tau} \int_0^t m(s) ds.$$

A uniform hidden target is expected to be encountered when cumulative tested capacity reaches roughly $N/2$. This provides the correct conceptual baseline for dynamic spawning.

14.4 Lemma value versus worker value

For local control, use the static proxy

$$\hat{T}(N, m) = \frac{N\tau}{2m}.$$

Suppose a proposed fishing action is expected to eliminate ΔN candidate values at charged cost C_L . Its estimated time saving is

$$V_L = \frac{\Delta N \tau}{2m}.$$

Adding one nonduplicating search worker at charged cost C_W changes the proxy from m to $m + 1$, producing estimated time saving

$$V_W = \frac{N\tau}{2m(m+1)}.$$

The controller should compare normalized marginal values

$$\frac{V_L}{C_L} \quad \text{and} \quad \frac{V_W}{C_W}.$$

Thus lemma fishing is favored when

$$\Delta N > \frac{N}{m+1} \frac{C_L}{C_W},$$

while worker activation is favored when the inequality is reversed, subject to the clock, verifier, and activation rules of the declared experiment. Under equal marginal costs this threshold simplifies to

$$\Delta N > \frac{N}{m+1}.$$

The formula makes explicit the two different ways CDF can reduce expected settlement time:

$$\boxed{\text{lemma fishing decreases } N, \quad \text{worker activation increases } m.}$$

14.5 Basin-mode control

Once $\text{Basin}(t) = 1$, Player Zero should not infer that brute force has been mathematically proved to be the only possible method. The justified conclusion is weaker: the admitted discovery mechanisms are currently producing negligible certified contraction and negligible reusable structural gain.

At each subsequent legal Player-Zero turn, the controller therefore re-evaluates V_L/C_L against V_W/C_W . If worker activation has the larger estimated marginal value, Player Zero should activate as aggressively as the declared rules permit. A Board alone is control state; it affects this comparison only when the frozen experiment explicitly couples that Board to added worker capacity. Under the strict spawn-credit architecture of the present paper, this means spending one available pending spawn credit at each eligible Player-Zero turn until the credits or resource budget are exhausted. A different experiment that allows basin-triggered activation without previously earned credits must declare that controller rule in advance rather than silently changing the baseline CDF architecture.

If persistent basin detection continues while fishing value approaches zero, the policy naturally shifts toward local exhaustive search. The important scientific claim is therefore not

“brute force is the only conceivable method,”

but rather

no admitted structural action has demonstrated positive frontier value

$\implies$

allocate increasing resources to certified local penetration

14.6 A compact Player-Zero fishing prompt

A practical controller prompt for this regime is:

Maintain the surviving hidden-parameter candidate set K_t and its size N_t . Do not enumerate candidates unless local exhaustive search has become the selected fallback. Measure recent certified frontier-contraction yield λ_t and reusable structural-gain yield ν_t . Declare a candidate basin only when both remain below their fixed thresholds for the required number of windows; do not use Board-overlap statistics. At every legal Player-Zero turn, estimate the expected candidate reduction from the next fishing action and compare its charged marginal value with the charged marginal value of activating one additional nonduplicating search worker. Spend the next available resource on whichever action gives the larger expected reduction in remaining settlement time. If persistent basin detection drives the estimated fishing value toward zero, spend available activation capacity aggressively at legal Player-Zero turns and move toward certified local exhaustive search. Never treat timeout, failure to find a proof, or mere lack of frontier contraction as a proof that a candidate region is impossible.

This prompt converts the informal question “Am I in a basin?” into a measurable controller decision while keeping all theorem-producing claims behind the fixed verifier.

15 Querying our way to an elusive proof: an Ocean-style blind proxy

The hidden-target controller admits a useful operational interpretation:

AMLD+0 queries its way toward an elusive proof.

The phrase is not intended to mean that Player Zero asks the verifier for the hidden answer. Rather, the federation repeatedly poses admissible structural questions—for example, ESLF prompts—whose successful outputs may reduce the surviving candidate region, expose a settlement basin, or alter the marginal value of additional search workers or declared Board-worker packages. The final proof remains verifier-gated. The queries change navigation and allocation, not theoremhood.

This viewpoint suggests a direct comparison with brute force. Brute force spends computation principally by increasing tested-candidate capacity. Fishing spends computation principally by asking whether certified structure can reduce the region that must later be searched. In the notation above,

brute force acts mainly on m , fishing acts mainly on N .

A hybrid AMLD+0 policy can do both: query while structural information has sufficient marginal value, then activate search workers aggressively once a basin has been detected and local penetration has become the better use of the remaining budget.

15.1 Reproducible blind hidden-target proxy

To test whether this mechanism can be a serious alternative to raw parallel enumeration, we ran a synthetic blind proxy inspired by the Ocean setting. It is deliberately *not* claimed to be an official NOTALD/Ocean benchmark evaluation. Its purpose is to isolate the information/computation tradeoff that the controller is meant to exploit.

The proxy is frozen as follows:

- hidden search bound $B = 10^8$;
- hidden target $b \sim \text{Unif}\{1, \dots, B\}$;
- hidden basin radius $W = \lfloor 10^U \rfloor$ with $U \sim \text{Unif}[3, 6]$;
- parallel worker count M sampled uniformly from $\{8, 16, 32, 64, 128\}$;
- one fishing query costs $\alpha = 100$ ordinary candidate-test time units and is serviced serially;
- each pre-signal probe is uniform on $\{1, \dots, B\}$ and independent of b ;
- a probe signals exactly when it lies in $[b - W, b + W] \cap \{1, \dots, B\}$;
- conditional on a signal, the successful probe is therefore uniform on that clipped hit interval; the signal reveals only that the target lies within distance W of the successful probe, not the target itself;
- the local phase searches the clipped interval $[\text{probe} - W, \text{probe} + W]$ in ascending order with M nonduplicating workers until b is reached;
- brute force searches $1, 2, \dots, B$ in ascending nonduplicating order with all M workers from the start;
- PRNG: NumPy 2.3.5 `Generator(PCG64)` with seed 20261001;
- Monte Carlo replication count $R = 200,000$.

The simulation file shipped with this paper, `cdf_ocean_proxy_v12.py`, is the executable specification. To avoid simulating potentially billions of failed probes one by one, it samples the number of queries Q directly from the mathematically equivalent geometric distribution with success probability

$$p = \frac{|[b - W, b + W] \cap \{1, \dots, B\}|}{B}.$$

It then samples the successful probe uniformly from the hit interval. Charged work and proxy wall time are

$$C_{\text{hybrid}} = \alpha Q + L, \quad T_{\text{hybrid}} = \alpha Q + \left\lceil \frac{L}{M} \right\rceil,$$

where L is the number of ascending local candidate tests required from the localized interval's lower endpoint to b . For brute force,

$$C_{\text{brute}} = b, \quad T_{\text{brute}} = \left\lceil \frac{b}{M} \right\rceil.$$

All search, query, and worker-time quantities in this proxy are therefore explicit.

The first replication under this exact specification produced

$$b = 17,160,399, \quad W = 239,862, \quad M = 64,$$

with $Q = 45$ charged fishing queries, successful probe 17,113,861, and $L = 286,401$ local candidate tests. Hence

$$C_{\text{hybrid}} = 290,901, \quad C_{\text{brute}} = 17,160,399,$$

and

$$T_{\text{hybrid}} = 8,976, \quad T_{\text{brute}} = 268,132.$$

This replication favors query-then-penetrate on both metrics, but it is only one draw.

Across $R = 200,000$ independently hidden instances under the same frozen rules, the aggregate results were:

Quantity	Parallel brute force	Query-then-penetrate
Mean charged work	49.982 million	0.870 million
Mean proxy wall-clock units	2.431 million	0.733 million
Fraction winning on charged work	—	99.14%
Fraction winning on proxy wall time	—	81.39%
Ratio of mean charged work	1	1/57.46
Ratio of mean wall time	1	1/3.32

Monte Carlo standard errors give the following approximate 95% intervals for the principal estimates:

$$\begin{aligned} \mathbb{E}[C_{\text{brute}}] &\in [49.856, 50.109] \text{ million,} \\ \mathbb{E}[C_{\text{hybrid}}] &\in [0.862, 0.877] \text{ million,} \\ \mathbb{E}[T_{\text{brute}}] &\in [2.418, 2.443] \text{ million,} \\ \mathbb{E}[T_{\text{hybrid}}] &\in [0.725, 0.740] \text{ million,} \\ \Pr(C_{\text{hybrid}} < C_{\text{brute}}) &\in [99.10\%, 99.18\%], \\ \Pr(T_{\text{hybrid}} < T_{\text{brute}}) &\in [81.22\%, 81.56\%]. \end{aligned}$$

These intervals quantify Monte Carlo sampling error only; they do not quantify model misspecification relative to a real theorem-proving benchmark.

The result does *not* show uniform domination of brute force. Conditioning on worker count in the same simulation gave:

M	Query-policy wall-time win rate	$\mathbb{E}[T_{\text{brute}}]/\mathbb{E}[T_{\text{hybrid}}]$
8	94.22%	8.40
16	89.68%	4.35
32	82.72%	2.15
64	74.67%	1.06
128	65.62%	0.53

The final row is especially important: the query policy wins a majority of individual $M = 128$ instances but has a worse *mean* wall time because rare long waits for a narrow hidden basin create a heavy tail. AMLD+0 should therefore be presented as a competitive alternative and adaptive complement to brute force, not as an unconditional replacement for it.

15.2 A simple crossover law

Away from domain boundaries, a uniformly placed probe lands in the radius- W basin with probability approximately $2W/B$. Hence the expected number of queries to obtain the first basin signal is approximately $B/(2W)$. If each query costs α candidate-time units, while local exhaustive penetration with M workers costs approximately W/M , then

$$\mathbb{E}[T_{\text{hybrid}}] \approx \alpha \frac{B}{2W} + \frac{W}{M}.$$

Parallel brute force has

$$\mathbb{E}[T_{\text{brute}}] \approx \frac{B}{2M}.$$

For $W \ll B$, the leading-order crossover is therefore

$$\boxed{W \gtrsim \alpha M.}$$

This law gives a direct interpretation to the controller questions. Fishing is attractive when a charged query is capable of identifying a basin whose effective radius is large compared with the product of query cost and available parallelism. Worker activation becomes increasingly attractive as the estimated basin shrinks; conversely, a large already-active M makes additional structural querying harder to justify under the crossover inequality.

Remark 15.1 (Status of the Ocean claim). The blind proxy supports the weaker and, at present, defensible showcase claim that AMLD+0 provides a serious alternative to brute force for Ocean-style hidden-target search when admissible queries reveal sufficiently wide structural basins. It does not establish that AMLD+0 is always at least as fast as brute force, and it does not constitute the missing frozen evaluator certificate for the official Ocean budget-frontier experiment. The experimental question for the actual Ocean family is therefore whether ESLF creates a basin signal wide enough, and cheaply enough, to satisfy the same marginal-value inequality under the benchmark’s frozen accounting rules.

16 Clocked-simulation antichains in AMLD+0

CDF is built inside a computational environment whose simulation geometry is already nontrivial. The theorem used here is imported from the dedicated AMLD+0 antichain construction [4]; we use the explicit permutation antichain exhibited by Spielman and Bóna [1]. Related simulation-degree context appears in earlier work [8]. The present section records the assumptions needed for the import and sketches the reflection mechanism so that the role of the theorem in CDF is explicit.

For each finite permutation $\pi \in S_n$, the cited construction defines a finite AMLD+0 system A_π . Its distinguished Pioneer frame has typed observable growth events ordered by

$$1 < 2 < \dots < n$$

and typed halting events whose relative order encodes

$$\pi(1), \dots, \pi(n).$$

Player Zero performs one common action at tick $t = 1$, activating the same fixed finite Board $\mathcal{B}^*$ in every witness. The Board carries no permutation-specific information and is thereafter rigid; the remaining non-Pioneer AMLD coordinates may be inert.

The imported reflection theorem depends on the following frozen rigidity conditions of that construction:

- (i) simulations respect the declared state types and distinguished Pioneer observations;
- (ii) one injective state recoding is used consistently across the complete Pioneer frame;
- (iii) all source observations are compared through one common nondecreasing unbounded macro-clock rather than trajectory-specific clocks;
- (iv) the Board and inert coordinates cannot encode a permutation-specific shortcut; and
- (v) the relative growth/halting observations used in the proof are preserved by admissible simulation.

These are not new axioms for arbitrary AMLD+0 systems; they are properties of the finite witness family used in the cited construction.

17 Permutation Reflection

Theorem 17.1 (Imported Permutation Reflection [4]). *For the finite witness systems A_π and A_σ constructed under the rigidity conditions above, if*

$$A_\pi \preceq_{\text{AMLD}+0} A_\sigma,$$

then

$$\pi \leq_{\text{pat}} \sigma,$$

where $\leq_{\text{pat}}$ denotes permutation-pattern containment.

Reflection sketch. Let an admissible simulation induce one state injection and one common macro-clock. Each distinguished source growth state must map to a distinct distinguished target growth state. If source trajectory i has grown while source trajectory k has not, the two typed observations must be simultaneously reproducible at a target stage; hence their target indices preserve the source growth order. Therefore the selected target indices satisfy

$$j_1 < j_2 < \cdots < j_n.$$

Likewise, if $\pi(i) < \pi(k)$, there is a source stage at which the i -trajectory has halted while the k -trajectory has not. Common-clock preservation of the typed halting observations forces

$$\sigma(j_i) < \sigma(j_k).$$

Applying the same comparison in the reverse ordering gives equivalence of relative order, so

$$\sigma(j_1), \dots, \sigma(j_n)$$

has pattern π . The full formalization of the witness states, typing, and rigidity clauses is the imported construction cited above. $\square$

18 AMLD+0 Infinite-Width Theorem

Spielman and Bóna proved that finite permutations under pattern containment contain an infinite antichain [1]. Choose a countable sequence

$$\pi_1, \pi_2, \dots$$

of pairwise incomparable finite permutations and construct the corresponding systems $A_r = A_{\pi_r}$.

Theorem 18.1 (Imported AMLD+0 Infinite Clocked-Simulation Antichain [4]). *The clocked-simulation degree poset of the scheduled AMLD+0 witness class contains a countably infinite antichain whose degrees are represented by finite, finitely describable AMLD+0 systems.*

The cited construction already works in a restricted fragment in which:

- (1) *Player Zero performs exactly one nontrivial action;*
- (2) *that action occurs at tick $t = 1$;*
- (3) *the same fixed Board is activated in every witness;*
- (4) *the Board carries no permutation-specific information;*
- (5) *the Board is thereafter rigid;*
- (6) *only the Pioneer role has nontrivial dynamics; and*
- (7) *the other AMLD roles and BANK coordinates are inert.*

Therefore the degree poset has width at least $\aleph_0$.

Proof. Take a countably infinite permutation antichain $\{\pi_r : r \geq 1\}$. If $A_r \preceq_{\text{AMLD+0}} A_s$, imported Permutation Reflection yields $\pi_r \leq_{\text{pat}} \pi_s$, contradicting incomparability. The reverse simulation is excluded symmetrically. Hence the degrees represented by the finite systems A_r are pairwise incomparable. $\square$

19 Why the antichain matters for CDF

This result is more than background. It establishes that even extremely restricted AMLD+0 systems need not lie on a single linear scale of simulation strength.

Thus CDF should not generally expect discovery states to satisfy

$$X_1 \preceq X_2 \preceq X_3 \preceq \dots$$

under one total notion of computational power. The ambient space can contain incomparable directions.

This gives a structural justification for a federation. Different Scouts or ATPs may explore genuinely incomparable regions rather than merely producing slower or faster versions of the same computation.

The result also distinguishes two claims:

Board presence

and

adaptive Board information.

The antichain exists even when the Board is fixed and informationally inert. CDF begins where that theorem stops: Player Zero is permitted to respond to certified settlement information and to alter future discovery capacity.

20 A first CDF research program

The architecture suggests several immediate theorem targets.

20.1 Scout conservativity

Prove that replacing AMLD roles by AMLD-capable Scouts does not alter theoremhood under a fixed verifier.

20.2 AMLD+0 Scout simulation

Determine conditions under which a Scout can emulate not only P, R, I, D, C but also the relevant Player-Zero control state.

20.3 Frontier correctness

Give conditions under which the analytical measure r_t and the discrete credit rank $\gamma_t = \kappa(F_t)$ are verifier-computable, and under which a new record in γ_t genuinely represents certified structural progress.

20.4 Ordinary versus Graduate Scout activation

Determine, at a legal Player-Zero service turn with a pending spawn credit, when Player Zero should activate an ordinary Scout and when previously certified state should be inherited by a Graduate Scout. A new-record contraction of the frozen discrete credit rank creates the credit but does not force either choice.

20.5 ESLF productivity

For a fixed benchmark, compare

direct ATP search

against

ATP + ESLF + AMLD settlement + Scout feedback.

20.6 Adaptive value theorem

The strongest target is a theorem showing conditions under which clock-respecting, frontier-credit-triggered adaptive CDF has strictly lower expected charged settlement cost than its corresponding frozen federation.

Such a theorem would connect

order theory, verification, formal discovery, resource control.

21 Central Principle

The architecture may be summarized by one principle:

Certified progress earns the right to grow.

Discovery resources are not activated merely because the system has worked longer. A verifier-authorized new record in the frozen discrete credit rank may instead earn a spawn credit. Actual activation occurs only on a Player-Zero turn and only if Player Zero elects to spend that credit.

Likewise, ATP conjectures are not trusted merely because a powerful prover generated them. They become knowledge only after settlement and verification.

Thus CDF separates three functions:

$$\boxed{\text{generation} \neq \text{settlement} \neq \text{certification.}}$$

Player Zero operates above those functions as a controller rather than as an oracle.

22 Conclusion

CDF organizes ATPs, AMLD roles, Scouts, certified BANK memory, settlement geometry, and Player-Zero control into one feedback architecture.

Its proposed basic cycle is

$$\boxed{\begin{aligned} &\text{generate} \rightarrow \text{settle} \rightarrow \text{verify} \rightarrow \text{new record credit-rank contraction} \rightarrow \text{earn spawn credit} \\ &\rightarrow \text{Player-Zero turn} \rightarrow \text{optional activation} \rightarrow \text{generate again} \end{aligned}}$$

Under the AMLD-capable Scout hypothesis, a single Scout can emulate every ordinary AMLD role and can therefore execute any finite AMLD-compatible ESLF procedure.

What remains unresolved is whether the same Scout can reproduce the full +0 layer and, when a new-record credit-rank contraction has earned a spawn credit, whether Player Zero should spend that credit on an ordinary Scout, a Graduate Scout, a Board, or another declared activation target at a later legal Player-Zero turn.

The incorporated AMLD+0 antichain theorem supplies a complementary structural fact: even finite, highly restricted AMLD+0 systems already exhibit infinite simulation width. CDF therefore operates in a discovery geometry in which incomparable computational forms arise naturally.

The longer-term objective is not merely to run more theorem provers. It is to construct a federation in which verifier-certified discoveries alter the geometry of the remaining problem, that altered geometry controls the deployment of new discovery processes, and those processes generate new conjectures for further certified settlement.

In that sense, CDF is intended as a model of *adaptive, verifier-gated formal discovery*.

A Installing an AMLD+0 Reference Instance

A.1 What “install AMLD+0” means

AMLD+0 is presently an architecture and protocol rather than a claim that one canonical AMLD+0 executable must exist. Accordingly, to *install an AMLD+0 instance* means to instantiate the required components, connect them through declared interfaces, and demonstrate by self-test that the verifier trust boundary and the Player-Zero clock discipline are actually enforced.

A minimal installation contains

$$\boxed{P, R, I, D, C, V, \text{BANK}, \mathcal{B}, P_0, \chi}$$

where P, R, I, D, C are the five AMLD roles, V is the verifier, BANK is persistent verifier-authorized memory, $\mathcal{B}$ is Board state, P_0 is Player Zero, and χ is the global service schedule. Scout count,

Board count, and worker count are separate coordinates whenever those resources are enabled. ATPs, Scouts, Graduate Scouts, ML guidance, ESLF, and adaptive spawning are optional extensions unless the experiment being reproduced requires them.

A.2 Before installation: freeze the experiment

Before software is launched, write down the rules of the run. At minimum declare:

- (1) the formal language and theorem/proof format;
- (2) the verifier V and the exact condition under which V accepts a certificate;
- (3) which information each role may read;
- (4) which state each role may propose changes to;
- (5) the global clock or service policy χ ;
- (6) the powers reserved to Player Zero;
- (7) the persistent Board and BANK formats;
- (8) all charged resources, including search, verification, training, memory, communication, and spawning when applicable;
- (9) whether Scouts, Graduate Scouts, ATP conjecture generation, ESLF, or ML guidance are enabled;
- (10) any anti-shortcut, hidden-information, or benchmark-specific restrictions.

This frozen specification is part of the installation because otherwise two implementations called “AMLD+0” may silently be running different protocols.

A.3 Minimal component layout

An implementation may use processes, threads, services, agents, containers, or another effective representation. The following logical modules are sufficient for a minimal reference instance:

V : Verifier

Reads a proposed certificate and returns accept or reject. It is the only component authorized to promote theorem-producing claims into certified BANK.

BANK: Certified memory

Stores only verifier-authorized formal results together with enough provenance to identify the certificate and verifier decision that admitted them.

B : Board Stores persistent shared control or search state that is legal for the declared experiment. Board state is not automatically certified theoremhood.

P : Prover Searches for proof certificates.

R : Refuter

Searches for refutation or counter-certificates appropriate to the formal environment.

***I*: Independence role**

Investigates independence or inability to derive either side under the admitted formal rules when such tasks are meaningful.

***D*: Dependence role**

Searches for reductions, implications, dependencies, or structural relations.

***C*: Contradiction role**

Searches for contradiction certificates or inconsistency witnesses under the declared proof system.

P_0 : Player Zero

Schedules, allocates, activates, or selects policies only within its declared authority. Player Zero is a controller, not a theorem oracle.

χ : Clock/scheduler

Determines whose service turn occurs at each global tick and therefore when Player Zero may legally take Player-Zero-only actions.

A.4 Reference installation procedure

The following procedure is deliberately implementation-neutral so that AMLD+0 can be reproduced in different programming languages and proof environments.

Install 1. Install or choose the formal verifier. Choose a proof checker or formal verifier appropriate to the target language. Confirm that it can be invoked deterministically on a candidate certificate and returns a machine-readable accept/reject result.

Install 2. Create certified BANK. Initialize BANK empty except for the frozen admitted starting theory. Require that theorem-producing additions pass through one write gate controlled by V . No role, ATP, Scout, or Player Zero should have an alternate path for inserting uncertified theoremhood claims.

Install 3. Create the five AMLD role workers. Instantiate P, R, I, D, C . Give each worker the input and output interface declared by the experiment. A worker may propose a certificate, conjecture, dependency, contradiction, or other legal object, but it does not decide its own certification status.

Install 4. Create persistent Board state. Initialize $\mathcal{B}$ with the declared base control state. Keep Board state logically distinct from BANK: the Board may remember schedules, search locations, policy state, queues, budgets, or other control information without thereby asserting that those items are mathematical theorems.

Install 5. Install Player Zero. Instantiate P_0 with exactly the declared scheduling and activation authority. If spawning is enabled, Player Zero must not spawn merely because another role has discovered something; it may spawn only under the frozen new-record credit-rank and clock rules. The implementation must update Scout, Board, and worker counters separately.

Install 6. Install the global service clock. Implement $\chi(t)$. Every service event should identify the current tick, the component receiving service, the charged work used, and any state transition produced. Player-Zero-only actions must be rejected when $\chi(t) \neq P_0$.

Install 7. Connect proposals to settlement and verification. Route ATP or Scout proposals into untrusted queues. Route theorem-producing certificates to V . Only accepted certificates may change certified BANK. Record rejects as search history if the protocol permits, but never as certified knowledge.

Install 8. Enable optional discovery modules. If the experiment needs ATP conjecture generation, ordinary Scouts, Graduate Scouts, ESLF, or ML guidance, connect them only after the minimal verifier/BANK/clock path works. A Graduate Scout may inherit a declared learned or certified state; a fresh Scout receives only its declared base initialization. Training and inherited-model costs must be accounted for according to the benchmark rules.

Install 9. Turn on logging and accounting. For reproducibility, log at least: global tick; active role; input state identifier; proposal; verifier decision; BANK change; Board change; Player-Zero action; spawn-credit change; resource cost; and random seed when randomness is used.

Install 10. Run the AMLD+0 installation self-test. Do not call the installation complete until the tests below pass.

A.5 Required self-test

A minimal self-test should use a tiny formal problem for which one valid and one invalid certificate are known.

Test 1: valid certificate enters BANK. Ask P to submit a known valid certificate π for a simple statement L . Require

$$V(L, \pi) = 1 \implies L \in \text{BANK}.$$

The log should show the proposal, verification event, and BANK admission.

Test 2: invalid certificate cannot enter BANK. Submit an intentionally malformed or invalid certificate π' . Require

$$V(L, \pi') = 0 \implies L \notin \text{BANK}.$$

This test should fail the installation if any alternate component can insert L as certified knowledge.

Test 3: Player-Zero clock discipline. Attempt a Player-Zero-only action at a tick t with $\chi(t) \neq P_0$. The system must reject or defer the action. Repeat at a legal Player-Zero tick and confirm that the action is processed according to the frozen policy.

Test 4: Board/BANK separation. Write an unverified search note or candidate conjecture to Board state. Confirm that it does not appear in certified BANK until a corresponding certificate is accepted by V .

Test 5: restart persistence. Restart the instance. Certified BANK and the declared persistent Board state should reload with provenance intact, while ephemeral role-local state should behave according to the frozen restart policy.

A.6 A minimal run after installation

After the self-test, a first AMLD+0 run can be extremely small:

- (1) load a tiny theorem target T and frozen hypotheses H ;
- (2) place $\text{Settle}(T)$ into the work queue;
- (3) allow P, R, I, D, C to receive turns according to χ ;
- (4) require all theorem-producing claims to pass through V ;
- (5) let Player Zero perform only the control actions authorized at its turns;
- (6) stop when a declared settlement certificate is accepted or the charged budget is exhausted;
- (7) export the event log, verifier certificates, final BANK, final Board state, and configuration used for the run.

A.7 In plain English

To install AMLD+0, first choose the proof checker that decides what counts as formally certified. Then create one protected memory area, BANK, that only the checker is allowed to certify into. Start five workers with different jobs: try to prove, refute, test independence, find dependencies, and find contradictions. Give them a shared Board for legal working memory, but do not confuse Board notes with proven facts. Add Player Zero as the scheduler/controller and give the whole system a clock so that each worker, including Player Zero, acts only on its legal turns. Finally, test the installation with one proof that must be accepted, one bad proof that must be rejected, and one Player-Zero action attempted at the wrong time. If an unverified claim can enter BANK, or Player Zero can bypass the clock, the AMLD+0 installation is not correct.

A.8 Installation checklist

A reference AMLD+0 installation is ready for experiments only when all of the following are true:

- P, R, I, D, C are instantiated and callable;
- V is fixed and independently invoked on theorem-producing certificates;
- BANK rejects uncertified theoremhood claims;
- Board state is distinct from BANK;
- Player Zero has an explicit authority boundary;
- χ enforces legal service turns;
- resource accounting and seeds are recorded;
- restart behavior is declared;
- optional ATPs, Scouts, Graduate Scouts, ESLF, and ML components cannot bypass V ;
- the valid-certificate, invalid-certificate, clock, Board/BANK, and persistence self-tests pass.

References

- [1] D. A. Spielman and M. Bóna, “An Infinite Antichain of Permutations,” *The Electronic Journal of Combinatorics* 7 (2000), No. N2.
<https://doi.org/10.37236/1540>
- [2] Alan Bundy, A. Smaill, and J. Hesketh, “Turning Eureka Steps into Calculations in Automatic Program Synthesis,” in *Proceedings of the UK IT 90 Conference*, 1990.
University of Edinburgh Research Explorer
- [3] Andrew Ireland and Alan Bundy, “Using Failure to Guide Inductive Proof,” University of Edinburgh, Department of Artificial Intelligence, 1992.
University of Edinburgh Research Explorer
- [4] Brian Tenneson, “Clocked-Simulation Antichains in AMLD+0: Player-Zero Board Activation and a Direct Finite Construction,” 2026.
- [5] Brian Tenneson, “Password-Seeking with AMLD+0 — Singleton Enumeration, SAT, and CNF Pruning,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/password_seeking_with_amld_plus_0_singleton_enumeration_sat_cnf_pruning/
- [6] Brian Tenneson, “AMLD+0: Lineage and Foundational Architecture,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/amld_plus_0_part_i_section_1_lineage_and_foundational_architecture/
- [7] Brian Tenneson, “A Personal Formal-Discovery Conjecture and Its Settlement,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/personal_formal_discovery_conjecture_and_its_settlement/
- [8] Brian Tenneson, “Depths of Induction and Formalized Self-Awareness — Version 49,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/depths_of_induction_v49/
- [9] Brian Tenneson, “Fundamental Theorem of Round-Robin AMLD Settlement — Finite-Settlement Completeness, GBG Reachability, CSG Horizons, and the No-Uniform-Deadline Boundary,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/fundamental_theorem_round_robin_amld_settlement_002/
- [10] Brian Tenneson, “Preorders and Fair Settlement Geometry — SIC/AMLD Simulation Degrees and a High-Probability Complexity Framework for SAT,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/preorders_and_fair_settlement_geometry_001/
- [11] Brian Tenneson, “Five-Way Pac-Man — Cooperative Search Geometry for Automated Metalogical Deciders,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/five_way_pac_man_amld_003/
- [12] Brian Tenneson, “Automated Metalogical Deciders II — Decision Geometry, Verified BANK Compression, and Five-Role Cooperative Settlement Theory,” 2026.

- https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/automated_metalogical_deciders_ii_decision_geometry_002/
- [13] Brian Tenneson, “Universal Settlement Horizons — Canonical Minimum AMLD Certificates, Finite Breakthrough Ladders, and the Noncomputability of the Last Breakthrough,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/universal_settlement_horizons_002/
 - [14] Brian Tenneson, “AMLDs as Cartographers, Pioneers, and Omnicartographers,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/amlds_cartographers_pioneers_omnicartographers/
 - [15] Brian Tenneson, “Geodesic Continuation and Multiresolution Omnicartography,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/geodesic_continuation_and_multiresolution_omnicartography/
 - [16] Brian Tenneson, “Metalogical Cartography I — AMLD-5, Verified BANK Geometry, and Settlement Acceleration,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/metalogical_cartography_i/
 - [17] Brian Tenneson, “Verifier Certificate 001 — I Know That I Know That I Am an AMLD,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/verifier_certificate_001/
 - [18] Brian Tenneson, “Security by Separation, Verification, and Settlement: An AMLD-Oriented Mathematical Treatise,” 2026.
https://btenneson.github.io/pub/cs.CR_Cryptography_and_Security/amld_security_treatise_v0_1/
 - [19] Brian Tenneson, “Recursive “What If?” Imagination in AMLD,” 2026.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/recursive_what_if_imagination_in_amld/

Suggested Further Reading

For readers who want broader context on the surrounding AMLD/CDF research program, the following companion works are especially relevant. They are not required for the formal claims of this paper, but they develop closely related ideas in cooperative proof search, omnicartography, epistemic handoff, learned guidance, and federated verified memory.

- Brian Tenneson, *Depths of Induction: ML-Guided Proof Search, Cooperative AMLDs, and Formalized Self-Awareness — Version 48*.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/depths_of_induction_v48/
- Brian Tenneson, *Oracle and Omnicartographer Theory — A Running Mathematical Development*.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/oracle_omnicartographer_theory_003/

- Brian Tenneson, *The Self Aware Machine: Epistemic Handoff and Successor-Safe Self-Awareness*.
https://btenneson.github.io/pub/cs.L0_Logic_in_Computer_Science/self_aware_machine_epistemic_handoff/
- Brian Tenneson, *AMLD Security 3: Eight Formal Application Sketches for Intelligent Target Settlement*.
https://btenneson.github.io/pub/cs.CR_Cryptography_and_Security/amld_security_3/
- Brian Tenneson, *AMLD Security 4: Federated Verified Memory Across Eight Application Domains*.
https://btenneson.github.io/pub/cs.CR_Cryptography_and_Security/amld_security_4/